WEEK 8: DATABASE RELATIONSHIP ANALYSIS USING JOINS

Project: Saree Boutique Management System

Scenario

The Saree Boutique Management System stores product, category, order, and payment information in separate tables. The management team wants to combine related information to understand customer purchases, order details, and payment status.

1. Implement INNER JOIN

Concept: INNER JOIN returns only matching records from both tables.

Task 1.1 – Display customer details along with their orders

```
SQL> SELECT DISTINCT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      O.ORDER_ID,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT,
  7      O.ORDER_STATUS
  8  FROM ORDERS O
  9  INNER JOIN ORDER_DETAILS OD
 10      ON O.ORDER_ID = OD.ORDER_ID;

CUSTOME CUSTOMER_NAME                                      ORDER_I
------- -------------------------------------------------- -------
ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------------------ ------------ --------------------
CUS0001 YAMINI                                             ORD0001
24-SEP-26                 25000 Pending

CUS0002 NANDY                                              ORD0002
24-SEP-26                 15000 Shipped
```

Task 1.2 – Display order details with payment information

```
SQL> SELECT
  2      O.ORDER_ID,
  3      O.CUSTOMER_NAME,
  4      O.ORDER_DATE,
  5      O.TOTAL_AMOUNT,
  6      P.PAYMENT_MODE
  7  FROM ORDERS O
  8  INNER JOIN PAYMENT P
  9      ON O.ORDER_ID = P.ORDER_ID;

ORDER_I CUSTOMER_NAME                                      ORDER_DATE
------- -------------------------------------------------- ------------------
TOTAL_AMOUNT PAYMENT_MODE
------------ --------------------
ORD0001 YAMINI                                             24-SEP-26
       25000 UPI

ORD0002 NANDY                                              24-SEP-26
       15000 Card

ORD0002 NANDY                                              24-SEP-26
       15000 UPI


ORDER_I CUSTOMER_NAME                                      ORDER_DATE
------- -------------------------------------------------- ------------------
TOTAL_AMOUNT PAYMENT_MODE
------------ --------------------
ORD0002 NANDY                                              24-SEP-26
       15000 Cash

ORD0001 YAMINI                                             24-SEP-26
       25000 UPI
```

Task 1.3 – Retrieve products purchased by customers

SELECT

```
SQL> SELECT
  2      O.CUSTOMER_NAME,
  3      O.ORDER_ID,
  4      P.PRODUCT_ID,
  5      P.PRODUCT_NAME,
  6      OD.QUANTITY,
  7      P.PRICE
  8  FROM ORDERS O
  9  INNER JOIN ORDER_DETAILS OD
 10      ON O.ORDER_ID = OD.ORDER_ID
 11  INNER JOIN PRODUCT1 P
 12      ON OD.PRODUCT_ID = P.PRODUCT_ID;

CUSTOMER_NAME                                      ORDER_I PRODUCT
-------------------------------------------------- ------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
YAMINI                                             ORD0001 P101
Shoe
         2        378

YAMINI                                             ORD0001 P111
Sketch
         1         28

CUSTOMER_NAME                                      ORDER_I PRODUCT
-------------------------------------------------- ------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------

NANDY                                              ORD0002 P102
T-Shirt
         2        599

NANDY                                              ORD0002 P001
Kanchipuram Silk Saree

CUSTOMER_NAME                                      ORDER_I PRODUCT
-------------------------------------------------- ------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
         1       8500

NANDY                                              ORD0002 P002
Cotton Saree
         2       1800
```

2. Implement LEFT JOIN

Task 2.1 – Display all customers including customers who have not placed orders

```
SQL> SELECT DISTINCT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      O.ORDER_ID,
  5      O.ORDER_DATE,
  6      O.ORDER_STATUS
  7  FROM ORDERS O
  8  LEFT JOIN ORDER_DETAILS OD
  9      ON O.ORDER_ID = OD.ORDER_ID;

CUSTOME CUSTOMER_NAME                                      ORDER_I
------- -------------------------------------------------- -------
ORDER_DATE         ORDER_STATUS
------------------ --------------------
CUS0001 YAMINI                                             ORD0001
24-SEP-26          Pending

CUS0002 NANDY                                              ORD0002
24-SEP-26          Shipped
```

Note: Since there is no separate CUSTOMER table, this displays customers recorded in ORDERS. It cannot show customers who have never had an order.

Task 2.2 – Find customers without any purchases

```
SQL> SELECT DISTINCT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      O.ORDER_ID
  5  FROM ORDERS O
  6  LEFT JOIN ORDER_DETAILS OD
  7      ON O.ORDER_ID = OD.ORDER_ID
  8  WHERE OD.ORDER_ID IS NULL;

no rows selected
```

Note: This finds orders without matching order-detail rows, not customers who have never placed an order.

Task 2.3 – Display all products including products with no sales

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      P.PRICE,
  5      OD.ORDER_ID,
  6      OD.QUANTITY
  7  FROM PRODUCT1 P
  8  LEFT JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE ORDER_I   QUANTITY
---------- ------- ----------
P101
Shoe
       378 ORD0001          2

P111
Sketch
        28 ORD0001          1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE ORDER_I   QUANTITY
---------- ------- ----------

P102
T-Shirt
       599 ORD0002          2

P001
Kanchipuram Silk Saree

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE ORDER_I   QUANTITY
---------- ------- ----------
      8500 ORD0002          1

P002
Cotton Saree
      1800 ORD0002          2
```

3. Implement RIGHT JOIN

Concept: RIGHT JOIN returns all records from the right table and matching records from the left table.

Task 3.1 – Display all orders with customer information

```
SQL> SELECT DISTINCT
  2      O.ORDER_ID,
  3      O.CUSTOMER_ID,
  4      O.CUSTOMER_NAME,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT,
  7      O.ORDER_STATUS
  8  FROM ORDER_DETAILS OD
  9  RIGHT JOIN ORDERS O
 10      ON OD.ORDER_ID = O.ORDER_ID;

ORDER_I CUSTOME CUSTOMER_NAME
------- ------- ----------------------------------------
ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------------------ ------------ --------------------
ORD0002 CUS0002 NANDY
24-SEP-26                15000 Shipped

ORD0001 CUS0001 YAMINI
24-SEP-26                25000 Pending
```

Task 3.2 – Find orders where customer details are missing

```
SQL> SELECT
  2      ORDER_ID,
  3      CUSTOMER_ID,
  4      CUSTOMER_NAME,
  5      ORDER_DATE,
  6      TOTAL_AMOUNT
  7  FROM ORDERS
  8  WHERE CUSTOMER_ID IS NULL
  9     OR CUSTOMER_NAME IS NULL;

no rows selected
```

Task 3.3 – Display all payment records with order details

```
SQL> SELECT
  2      PM.PAYMENT_MODE,
  3      O.ORDER_ID,
  4      O.CUSTOMER_NAME,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT
  7  FROM ORDERS O
  8  RIGHT JOIN PAYMENT PM
  9      ON O.ORDER_ID = PM.ORDER_ID;

PAYMENT_MODE         ORDER_I CUSTOMER_NAME
-------------------- ------- ----------------------------------------
ORDER_DATE         TOTAL_AMOUNT
------------------ ------------
UPI                  ORD0001 YAMINI
24-SEP-26                 25000

UPI                  ORD0001 YAMINI
24-SEP-26                 25000

Cash                 ORD0002 NANDY
24-SEP-26                 15000


PAYMENT_MODE         ORDER_I CUSTOMER_NAME
-------------------- ------- ----------------------------------------
ORDER_DATE         TOTAL_AMOUNT
------------------ ------------
UPI                  ORD0002 NANDY
24-SEP-26                 15000

Card                 ORD0002 NANDY
24-SEP-26                 15000
```

5. Retrieve Complete Order Details

Task: Generate a complete order report containing customer name, product name, quantity, order date, total amount, and payment status.

```
SQL> SELECT
  2      O.CUSTOMER_NAME,
  3      P.PRODUCT_NAME,
  4      OD.QUANTITY,
  5      O.ORDER_DATE,
  6      O.TOTAL_AMOUNT,
  7      PM.PAYMENT_STATUS
  8  FROM ORDERS O
  9  INNER JOIN ORDER_DETAILS OD
 10      ON O.ORDER_ID = OD.ORDER_ID
 11  INNER JOIN PRODUCT1 P
 12      ON OD.PRODUCT_ID = P.PRODUCT_ID
 13  LEFT JOIN PAYMENT PM
 14      ON O.ORDER_ID = PM.ORDER_ID;

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
YAMINI
Sketch
         1 24-SEP-26                 25000 Success

YAMINI
Shoe
         2 24-SEP-26                 25000 Success

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------

NANDY
Cotton Saree
         2 24-SEP-26                 15000 Success

NANDY
Kanchipuram Silk Saree

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
         1 24-SEP-26                 15000 Success

NANDY
T-Shirt
         2 24-SEP-26                 15000 Success

NANDY

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
Cotton Saree
         2 24-SEP-26                 15000 Pending

NANDY
Kanchipuram Silk Saree
         1 24-SEP-26                 15000 Pending
```

```
CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------

NANDY
T-Shirt
         2 24-SEP-26                15000 Pending

NANDY
Cotton Saree
         2 24-SEP-26                15000 Success

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------

NANDY
Kanchipuram Silk Saree
         1 24-SEP-26                15000 Success

NANDY
T-Shirt

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
         2 24-SEP-26                15000 Success

YAMINI
Sketch
         1 24-SEP-26                25000 Success

YAMINI

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DATE         TOTAL_AMOUNT PAYMENT_STATUS
---------- ------------------ ------------ --------------------
Shoe
         2 24-SEP-26                25000 Success


13 rows selected.
```

6. Display Customer Purchase History

Task 6.1 – Display all products purchased by a customer

```
SQL> SELECT
  2      O.CUSTOMER_NAME,
  3      O.CUSTOMER_ID,
  4      P.PRODUCT_NAME,
  5      OD.QUANTITY,
  6      P.PRICE,
  7      O.ORDER_DATE
  8  FROM ORDERS O
  9  INNER JOIN ORDER_DETAILS OD
 10      ON O.ORDER_ID = OD.ORDER_ID
 11  INNER JOIN PRODUCT1 P
 12      ON OD.PRODUCT_ID = P.PRODUCT_ID
 13  WHERE O.CUSTOMER_ID = 'CUS0002';

CUSTOMER_NAME                                      CUSTOME
-------------------------------------------------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ORDER_DATE
---------- ---------- ------------------
NANDY                                              CUS0002
T-Shirt
         2        599 24-SEP-26

NANDY                                              CUS0002
Kanchipuram Silk Saree
         1       8500 24-SEP-26

CUSTOMER_NAME                                      CUSTOME
-------------------------------------------------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ORDER_DATE
---------- ---------- ------------------

NANDY                                              CUS0002
Cotton Saree
         2       1800 24-SEP-26
```

Task 6.2 – Find the total amount spent by each customer

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_AMOUNT_SPENT
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  ORDER BY TOTAL_AMOUNT_SPENT DESC;

CUSTOME CUSTOMER_NAME                                    TOTAL_AMOUNT_SPENT
------- -------------------------------------------------- ------------------
CUS0001 YAMINI                                                      25000
CUS0002 NANDY                                                       15000
```

Find the number of orders placed by each customer

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOME CUSTOMER_NAME                                    NUMBER_OF_ORDERS
------- -------------------------------------------------- ----------------
CUS0001 YAMINI                                                           1
CUS0002 NANDY                                                            1
```

Task 6.4 – Display the latest purchase details of customers

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      ORDER_ID,
  5      ORDER_DATE,
  6      TOTAL_AMOUNT,
  7      ORDER_STATUS
  8  FROM (
  9      SELECT
 10          O.*,
 11          ROW_NUMBER() OVER (
 12              PARTITION BY CUSTOMER_ID
 13              ORDER BY ORDER_DATE DESC, ORDER_ID DESC
 14          ) AS RN
 15      FROM ORDERS O
 16  )
 17  WHERE RN = 1;

CUSTOME CUSTOMER_NAME                                    ORDER_I
------- -------------------------------------------------- -------
ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
------------------ ------------ --------------------
CUS0001 YAMINI                                           ORD0001
24-SEP-26                 25000 Pending

CUS0002 NANDY                                            ORD0002
24-SEP-26                 15000 Shipped
```

7. Generate Multi-Table Business Reports

Report 1 – Customer Order Report

Display: Customer name, order ID, order date, and order status.

```
SQL> SELECT
  2      CUSTOMER_NAME,
  3      ORDER_ID,
  4      ORDER_DATE,
  5      ORDER_STATUS
  6  FROM ORDERS
  7  ORDER BY ORDER_DATE DESC;

CUSTOMER_NAME                                    ORDER_I ORDER_DATE
------------------------------------------------- ------- ------------------
ORDER_STATUS
--------------------
NANDY                                            ORD0002 24-SEP-26
Shipped

YAMINI                                           ORD0001 24-SEP-26
Pending
```

Report 2 – Sales Report

Display: Product name, quantity sold, and total revenue.

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  4      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  5  FROM PRODUCT1 P
  6  INNER JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Kanchipuram Silk Saree
            1          8500

Cotton Saree
            2          3600

T-Shirt
            2          1198


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Shoe
            2           756

Sketch
            1            28
```

Report 3 – Payment Analysis Report

Display: Payment mode, number of transactions, and successful payments.

```
SQL> SELECT
  2      PAYMENT_MODE,
  3      COUNT(*) AS NUMBER_OF_TRANSACTIONS,
  4      SUM(
  5          CASE
  6              WHEN UPPER(PAYMENT_STATUS) = 'SUCCESS'
  7              THEN 1
  8              ELSE 0
  9          END
 10      ) AS SUCCESSFUL_PAYMENTS
 11  FROM PAYMENT
 12  GROUP BY PAYMENT_MODE;

PAYMENT_MODE         NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
-------------------- ---------------------- --------------------
Card                                      1                    1
Cash                                      1                    1
UPI                                       3                    2
```

Report 4 – Customer Purchase Analysis

4.1 Top purchasing customers

```
SQL> SELECT
  2      O.CUSTOMER_ID,
  3      O.CUSTOMER_NAME,
  4      SUM(OD.QUANTITY) AS TOTAL_ITEMS_PURCHASED
  5  FROM ORDERS O
  6  INNER JOIN ORDER_DETAILS OD
  7      ON O.ORDER_ID = OD.ORDER_ID
  8  GROUP BY O.CUSTOMER_ID, O.CUSTOMER_NAME
  9  ORDER BY TOTAL_ITEMS_PURCHASED DESC;

CUSTOME CUSTOMER_NAME                                    TOTAL_ITEMS_PURCHASED
------- -------------------------------------------------- --------------------
CUS0002 NANDY                                                                 5
CUS0001 YAMINI                                                                3
```

4.2 Customers with maximum orders

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(*) AS NUMBER_OF_ORDERS
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING COUNT(*) = (
  8      SELECT MAX(ORDER_COUNT)
  9      FROM (
 10          SELECT COUNT(*) AS ORDER_COUNT
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                      NUMBER_OF_ORDERS
------- -------------------------------------------------- ----------------
CUS0002 NANDY                                                             1
CUS0001 YAMINI                                                            1
```

4.3 Customers with highest spending

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) = (
  8      SELECT MAX(CUSTOMER_SPENDING)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_SPENDING
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                      TOTAL_SPENDING
------- -------------------------------------------------- --------------
CUS0001 YAMINI                                                      25000
```

Conclusion

In this practical, INNER JOIN, LEFT JOIN, and RIGHT JOIN were used to retrieve related information from the Saree Boutique database. Customer purchase history, complete order details, sales reports, and payment analysis were generated using SQL queries. These queries help analyze orders, products, and customer spending.